C++ 值传递 vs 传址调用（超清晰版）

下面用最直白、考试/写代码都能用的方式讲清楚。

一、基本概念

1. 值传递（传值调用）

- 把**实参的值复制一份**传给函数的形参
- 形参是**独立副本**，和外面的变量没关系

2. 传址调用

- 把**变量的内存地址**传给函数
- 函数通过指针操作**原来那个变量本身**

二、核心区别（直接背）

对比项	值传递	传址调用（指针）
传的是什么	变量的值（拷贝）	变量的地址
修改形参	不影响外面实参	**直接修改外面原变量**
内存开销	要拷贝数据，大对象效率低	只传地址（4/8字节），效率高
数组能否传递	不能真正传数组，会退化成指针	数组传参本质就是传址调用
语法	直接写变量	要取地址 `&`，函数里要用 `*` 解引用
安全性	高，不会误改原数据	可能误改原数据，可用 `const` 保护

三、代码对比一看就懂

1. 值传递（不会修改原数）

Code block

```
void add(int x) {
    x = x + 10;
}

int main() {
    int a = 5;
    add(a);
    // a 仍然是 5
}
```

2. 传址调用（会修改原数）

Code block

```
void add(int *x) {
    *x = *x + 10;
}

int main() {
    int a = 5;
    add(&a);
    // a 变成 15
}
```

四、适用场景（重点）

1. 什么时候用 **值传递**

- 只需要**读取数据**，不需要修改原变量
- 数据很小：int、double、char 等简单类型
- 希望函数安全，**绝对不影响外面变量**

示例：计算、打印、判断、查找等。

2. 什么时候用 **传址调用**

- 函数**需要修改外面的实参**（交换、修改数组、修改结构体）
- 传递**数组、大结构体、大对象**，避免拷贝浪费内存
- 需要“返回多个结果”时（C 语言常用）

典型例子：

- 交换两个数
- 对数组排序
- 修改结构体内容
- 函数需要传出多个值

五、特别提醒（数组必看）

C++ 里**数组不能值传递**。

你写：

Code block

```
void func(int arr[])
```

本质上等价于：

Code block

```
void func(int *arr)
```

也就是说：

数组传参 = 自动传址调用

函数里改数组元素，原数组一定会变。

六、一句话总结

- **值传递：传副本，不改原数据，安全但拷贝开销大。**
- **传址调用：传地址，改原数据，高效适合大对象/数组。**

如果你需要，我可以再帮你对比**传引用（&）**和这两种的区别，它在 C++ 里更常用。

（注：文档部分内容可能由 AI 生成）